

SECJ2203: Software Engineering

System Documentation (SD)

Project Title: PERWANIN System

Version 1.0

Date: 28/5/2022

School of Computing, Faculty of Engineering

Prepared by: GROUP ZEST

Revision Page

a. Overview

The current version of system documentation involved description of introduction and specific requirements. The introduction of this system documentation includes the purpose, scope, definitions, acronyms and abbreviations as well as reference. On the other hand, the interfaces, system features which consists of use cases, requirements, constraints, and software system attributes are explained and detailed in the specific requirement section.

b. Target Audience

Stakeholder

c. Project Team Members

Member Name	Role	Task	Status
Teh Jing Ling	Moderator	Monitor group progress	Complete
Zhu Yi Chen	Recorder	Record the results of discussion	Complete
Siew Yu Xuan	Skeptic	Give query and comment on the assignment	Complete
Eunice Lim Xian Ni	Accuracy checker & Reporter	Check and compile the assignment	Complete

d. Version Control History

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Teh Jing Ling	Completed Chapter 1 and 2, Section introduction and specific requirements	28/05/2022

Table of Contents

1	Introduction		3
	1.1	Purpose	3
	1.2	Scope	3
	1.3	Definitions, Acronyms and Abbreviations	4
	1.4	References	4
	1.5	Overview	5
2	Specific Requirements		6
	2.1	External Interface Requirements	6
		2.1.1 User Interfaces	6
		2.1.2 Hardware Interfaces	6
		2.1.3 Software Interfaces	6
		2.1.4 Communication Interfaces	7
	2.2	System Features	8
		2.2.1 UC001: Use Case Fill membership form	11
		2.2.2 UC002: Use Case Pay registration fee	13
		2.2.3 UC003: Use Case Receive notification	15
		2.2.4 UC004: Use Case Provide commitment readiness feedback	16
		2.2.5 UC005: Use Case Manage member's information	17
		2.2.6 UC006: Use Case Search member's information	19
		2.2.7 UC007: Use Case Manage to be organized programs	21
		2.2.8 UC008: Use Case Generate member's commitment reports	22
		2.2.9 UC009: Use Case Manage volunteers for each program	24
		2.2.10 UC010: Use Case Search for volunteers	25
		2.2.11 UC011: Use Case Send invitation notice	27
		2.2.12 UC012: Use Case Search program's information	28

		2.2.13	UC013: Use Case Receive invitation notice	30
		2.2.14	UC014: Use Case Access program information	31
		2.2.15	UC015: Use Case Make donations	32
		2.2.16	UC016: Use Case Manage financial information	34
		2.2.17	UC017: Use Case Generate financial documents	35
		2.2.18	UC018: Use Case Generate vouchers	37
		2.2.19	UC019: Use Case Generate financial report	38
		2.2.20	UC020: Use Case Generate receipt	40
		2.2.21	UC021: Use Case Manage donations	41
		2.2.22	UC022: Use Case Show collection progress	43
	2.3	Performance and Other Requirements		45
	2.4	Design Constraints		45
	2.5	Software System Attributes		45

1. Introduction

1.1 Purpose

The purpose of creating this System Documentation (SD) is to clarify the specific requirements and the features of the proposed hybrid system designed for PERWANIN. For this part, the documentation only includes the Specific Requirement Specification (SRS), which is used to describe how the software system should be developed. The SRS will also describe the functional and non-functional requirements that can fully satisfy the needs of the stakeholders, which is able to perform convenience system internal management and efficient public communication system in the proposed hybrid system of web-based and mobile. Based on the requirements, a set of use case and sequence diagram corresponding to their functions and features will also be included in this SRS documentation to describe the flow and interaction of the requirements of proposed PERWANIN system.

1.2 Scope

The software product is PERWANIN System to replace the current manual Perwanin website. PERWANIN System will benefit four types of users including member, secretary, treasurer and volunteer. To become a Perwanin member, PERWANIN System allows the public to fill in the membership form and pay the registration fee. After becoming a member of Perwanin, they can receive notification for each program to be organized. The member needs to provide commitment readiness feedback through the system once they decide to join any of the programs held. The system will then generate member's commitment report.

Besides, the system can be utilized by the secretary to search for members' information according to member's ID, name, expertise and interests. The system also allows the secretary to manage volunteers for each program including search for volunteers' information and send invitation notice. For treasurer, PERWANIN System allows them to manage financial information such as Perwanin's expenses and income. After that, the system will generate financial documents and allows them to manage donations from the donors. Moreover, the system enables the donors to access all the program details and make donations through it. PERWANIN System also provides functionality to volunteers where they can search for the program's information and receive invitation notice. However, the system does not include accounting functionality. Therefore, third party software such as general ledger system is required to perform accounting operation and track the PERWANIN System's financial data in real-time.

Our goal is to produce a hybrid system that can perform internal management and allow communication with the public. The system will be efficient, effective and user-friendly so that can improve the user experience.

1.3 Definitions, Acronyms and Abbreviation

Definitions of all terms, acronyms and abbreviation used are to be defined here.

SD – System Documentation

PERWANIN – Persatuan Wanita Prihatin Daerah Johor Bahru

NGO – Non-governmental organization

OTP – One-time password

PDPA – Personal Data Protection Act 2010 (PDPA)

1.4 References

MyHistoryFeed. (2019, Nov 13). *Software Requirements Specification (SRS Document)*.

Available at: <https://medium.com/trailblazer-of-salesforce/software-requirements-specification-srs-document-fd9ab103b18>

Smartdraw.com. (2022). *Activity Diagram - Activity Diagram Symbols, Examples, and More*.

Available at: <https://www.smartdraw.com/activity-diagram/#:~:text=An%20activity%20diagram%20visually%20presents,can%20be%20sequential%20and%20concurrent.>

Oleg C.. (2017, Dec 19). *A Brief Introduction to Domain Modeling*. Available at:

<https://olegchursin.medium.com/a-brief-introduction-to-domain-modeling-862a30b38353>

Lucidchart.com. (2022). *State Machine Diagram Tutorial*. Available at:

<https://www.lucidchart.com/pages/uml-state-machine-diagram#:~:text=A%20state%20diagram%2C%20sometimes%20known,a%20free%20Lucidchart%20account%20today!>

Amanda R.. (2022, Jan 23). *Design Constraints in Engineering: Definition & Example*.

Available at: <https://study.com/academy/lesson/design-constraints-in-engineering-definition-example.html>

Stephen W. (2017, Sep 18). *SRS: Software Requirement Specifications Basics*. Available at:

<https://www.bmc.com/blogs/software-requirements-specification-how-to-write-srs-with->

[examples/#:~:text=Software%20Requirements%20Specifications%2C%20also%20known,yo
u%20to%20your%20finished%20product.](#)

1.5 Overview

In this System Documentation (SD), we have included specific requirements needed for software development. Specific requirements in this project includes external and internal requirements, system features, performance and other requirements, design constraints and software system attributes. In external interface requirements, we have explained the user interfaces, hardware interfaces, software interfaces and communication interfaces. Every use case has provided use case descriptions and its sequence diagram in system features. Performance and other requirements, design constraints and software system attributes will be elaborated more in this project. All phases are documented with diagrams and descriptions.

2. Specific Requirements

2.1 External Interface Requirements

2.1.1 User Interfaces

a) Logical characteristics:

Standard Markup Language: HyperText Markup Language (HTML)

Stylesheet Language: Cascading Style Sheets (CSS)

Programming Language: JavaScript

b) Aspects

- First-time users must sign up to create an account before logging into the system.
- All users must login into the system before accessing particular webpages in the system.
- Users who do not login only can access limited webpage.
- User interface will displays the interface based on user role in the system.
- When the system has an error, the webpage will show an error message to the user.

2.1.2 Hardware Interfaces

Web server Ram: 32GB

Database server: 512GB

Internet bandwidth speed: 1 GB/s

Cables: Ethernet

System support pins: power, clock, reset, etc.

Network protocol: TCP/IP protocol

2.1.3 Software Interfaces

Required Software product

1. Name: MySQL Database management system

Mnemonic: MySQL

Specification number: N/A

Version number: v8.0

Source: <https://dev.mysql.com/doc/refman/8.0/en/mysql-nutshell.html>

2. Hostinger

Mnemonic: Hostinger

Specification number: N/A

Version number: v1.0

Source: <https://www.hostinger.com/>

3. Window 10

Name: Window 10

Specification number: 10240

Version number: 1903

Source: <https://docs.microsoft.com/en-us/windows/release-health/release-information>

Software interface

1. Acumatica Cloud ERP

The purpose of choosing Acumatica Cloud ERP as it is a cloud-based general ledger system. It easily integrates the tools and system the user already use. It can also improve efficiency with high user experience. It provides a low-code environment which eases our stakeholder when handling the transaction and payment. It comes along with dashboards, reporting tools, integrated document management, secure data and powerful customization tools.

2.1.4 Communication Interfaces

Local network protocols that applied are FTP and HTTPS protocols. FTP allows the users to transfer files from one machine to another where types of files may include program files, multimedia files, text files, and documents. In addition, another protocol which is HTTPS, is abbreviated as Hyper Text Transfer Protocol Secure. This is a standard protocol to secure communication between two computers where one of it using the browser and the another fetching data from the web server. Normally, HTTP is used for transferring data between the client browser and the web server. Thus, in the same case in HTTPS but in an encrypted format.

2.2 System Features

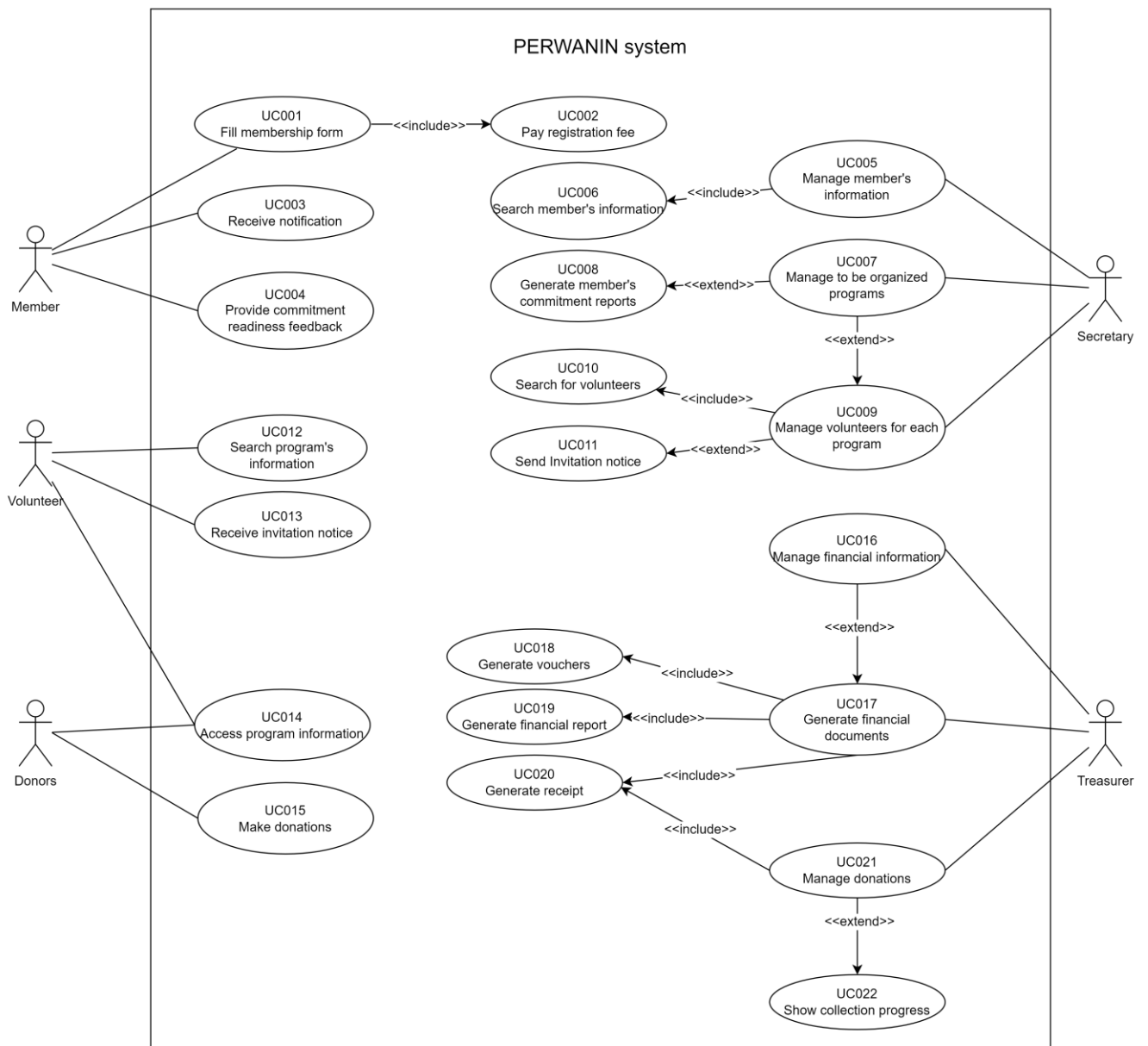


Figure 2.1: Use Case Diagram for PERWANIN System

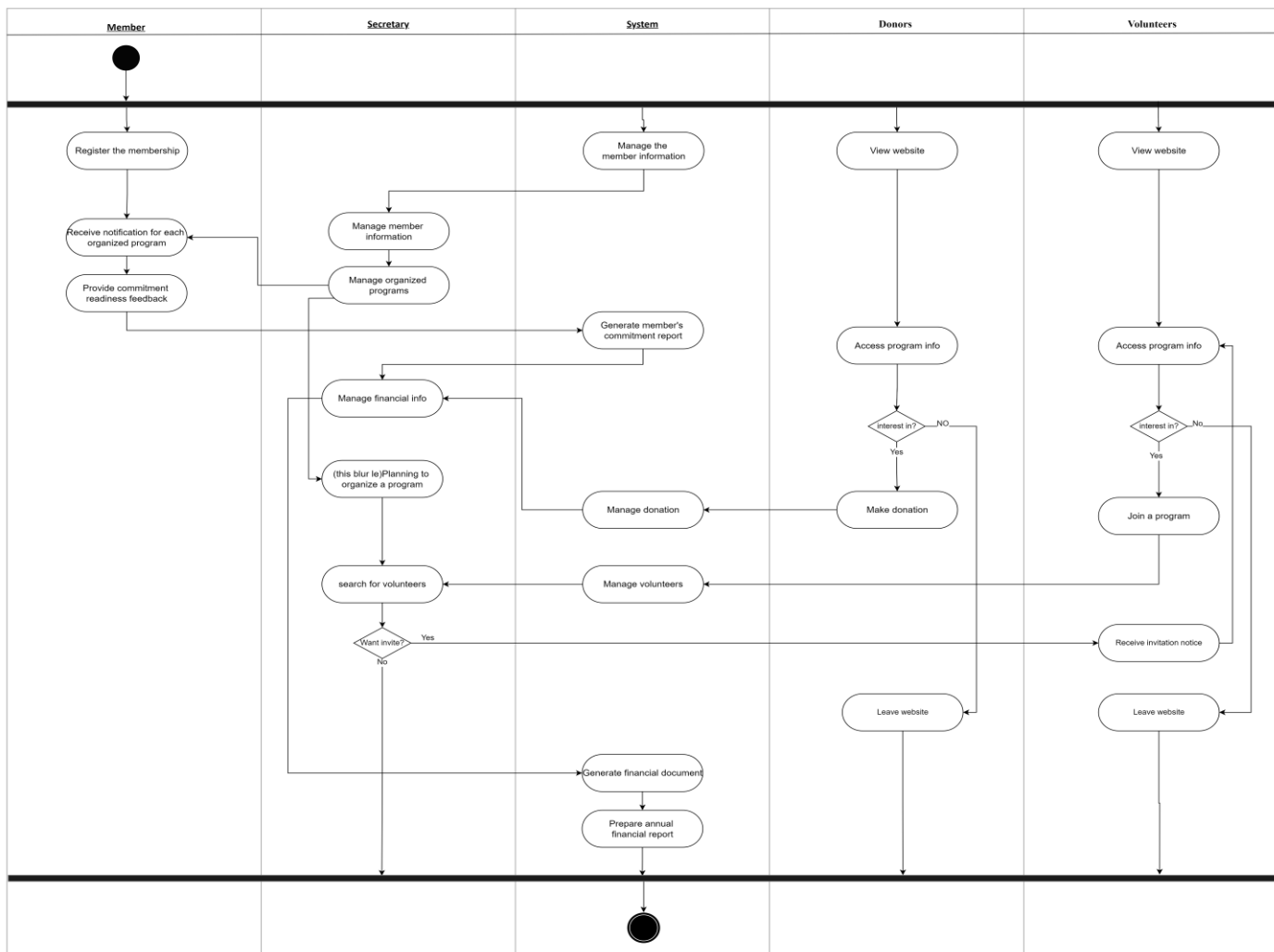


Figure 2.2: Activity Diagram for PERWANIN System

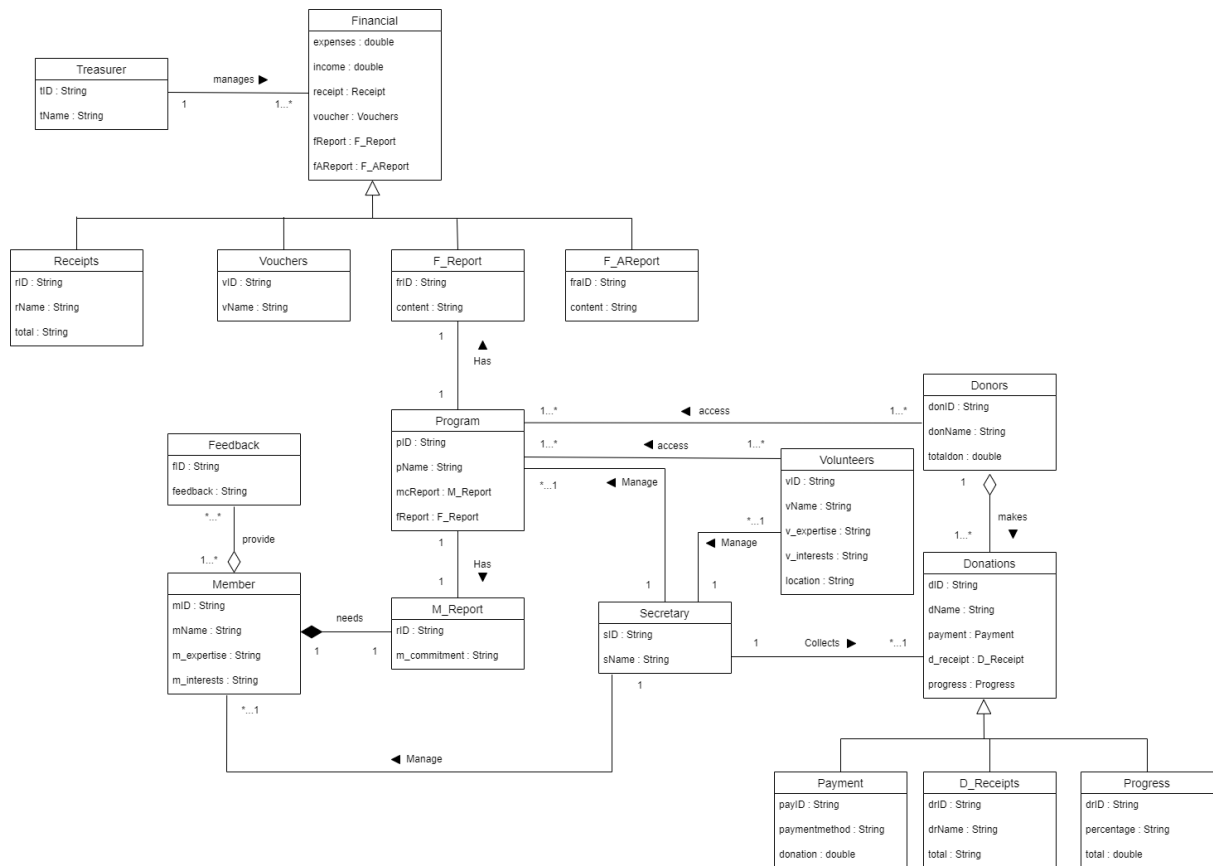


Figure 2.3: Domain Model for PERWANIN System

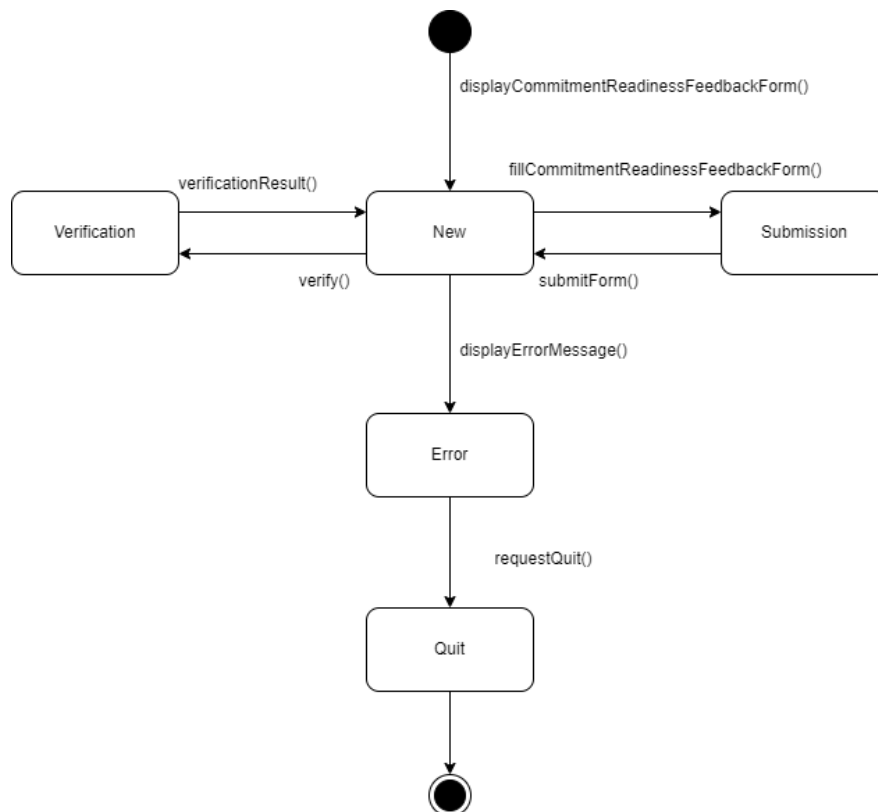


Figure 2.4: State Machine Diagram for Feedback Class

2.2.1 UC001: Use Case Fill in membership form

Table 2.1: Use Case Description for Fill in membership form

Use case: Fill in membership form
ID: UC001
Actors: Member
Preconditions: Member has registered an account and logged into the system.
Flow of events: 1. User selects the button “Apply as member” displayed at the dashboard of system. 2. The system passes the validation (EF1). 3. The system displays the empty membership form. 4. User fills in the membership form. 5. User submits the membership form. 6. The system passes the form validation (EF2). 7. System displays registration successful. 8. Use case ends.
Postconditions: Member information successfully stored in the database.
Alternative flow <i>n</i>: 1. User quits to fill in membership form 1.1 The system returns to the dashboard. 1.2 The use case is aborted.
Postconditions: Member information did not store in database.
Exception flow (if any): 1. User’s information already stored in database 1.1 The system displays error message on the screen, informing the user that he/she has filled in the membership before. 1.2 The use case is aborted. 2. System failed to insert new record into database 2.1 The system displays an error message on the screen. 2.2 Flow of events are executed again.

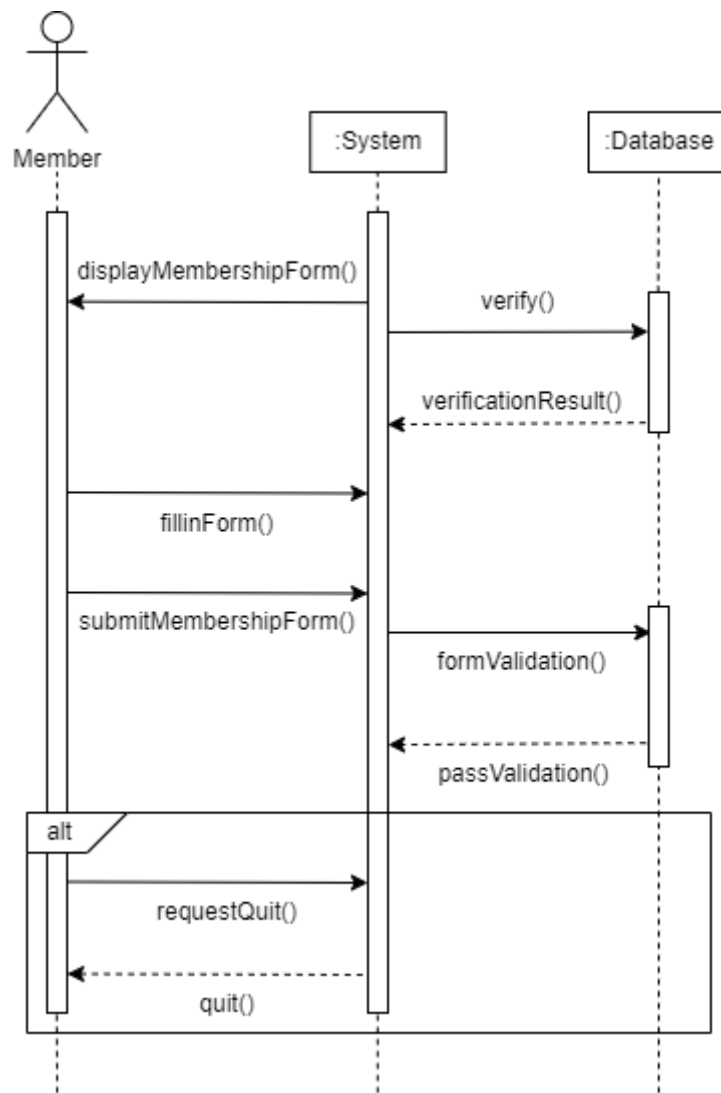


Figure 2.5: Sequence Diagram for Fill in membership form

2.2.2 UC002: Use Case Pay registration fee

Table 2.2: Use Case Description for Pay registration fee

Use case: Pay registration fee
ID: UC002
Actors: Member
Preconditions: Member has done filling in the membership form (UC001).
Flow of events: 1. The system displays the payment details for PERWANIN membership. 2. User enters the exact amount of money to pay. 3. System displays two payment methods: Credit/Debit card or FPX Online Banking. 4. User chooses either one payment method and fills in the banking details. 5. System sends OTP to user's phone and requests OTP from user. 6. User fills in OTP. 7. System passes the validation (EF1). 8. Payment status displays payment successful on screen. 9. User officially recognizes as PERWANIN member and has full access to the system. 10. Use case ends.
Postconditions: Member payment details successfully stored in the database of system.
Alternative flow <i>n</i>: 1. Member cancels the payment process 1.1 The system returns to membership form page. 1.2 The payment status remains showing not yet pay registration fee. 1.3 The use case is aborted.
Postconditions: 1. User is not officially a PERWANIN member and has limited access to the system.
Exception flow (if any): 1. Validation failed 1.1 The system displays an error message on the form. 1.2 Flow of events are executed again.

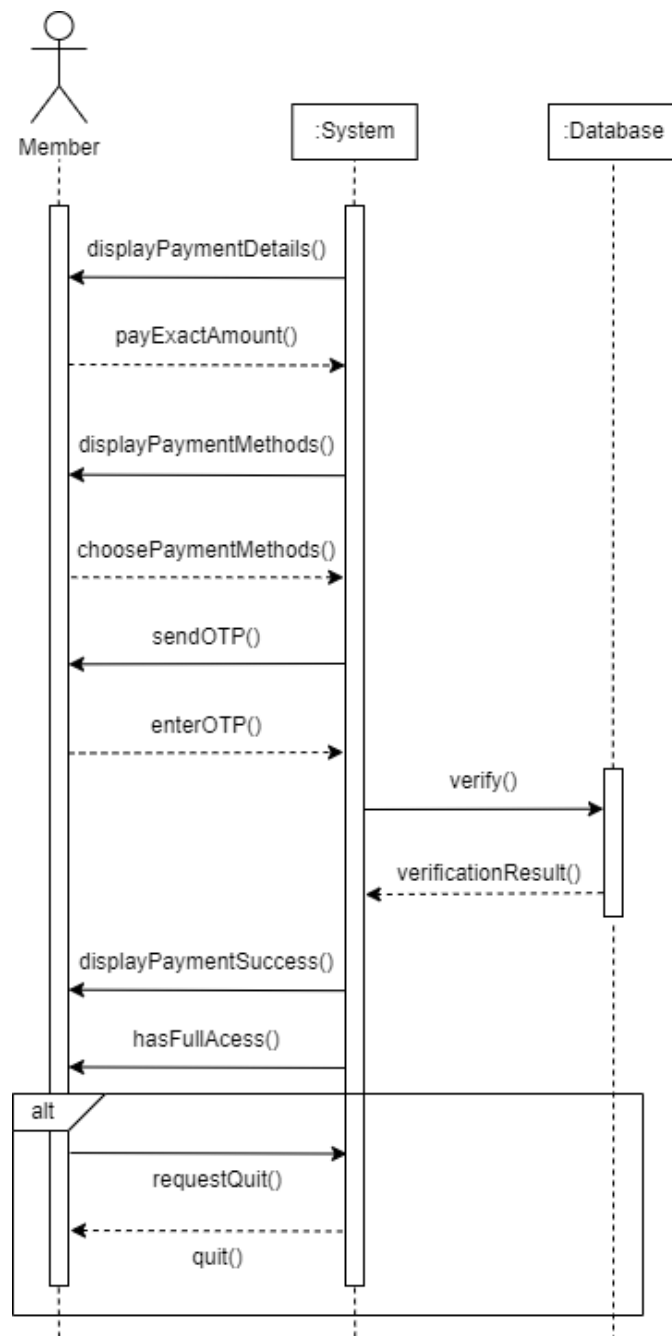


Figure 2.6: Sequence Diagram for Pay registration fee

2.2.3 UC003: Use Case Receive notification

Table 2.3: Use Case Description for Receive notification

Use case: Receive notification
ID: UC003
Actors: Member
Preconditions: Member logged into the system.
Flow of events: 1. The system pops up the notification for each program to be organized while member is browsing the system. 2. Member clicks the notification (EF1). 3. The system directs member to the page where it displays information about the program. 4. Use case ends.
Postconditions: The system collects the rate of number of members view and click the notification.
Alternative flow <i>n</i>: None
Postconditions: None
Exception flow (if any): 1. Cancel operation by user 1.1 The notification is closed. 1.2 The use case is aborted.

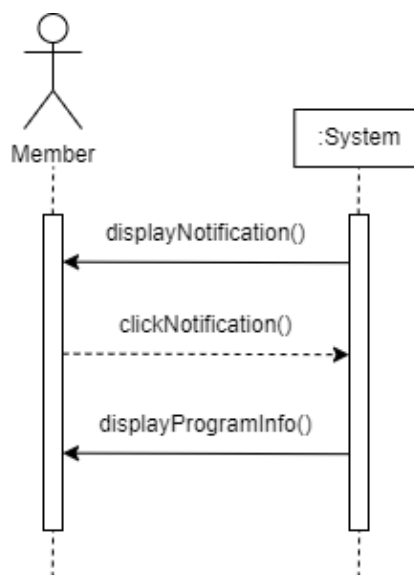


Figure 2.7: Sequence Diagram for Receive notification

2.2.4 UC004: Use Case Provide commitment readiness feedback

Table 2.4: Use Case Description for Provide commitment readiness feedback

Use case: Provide commitment readiness feedback
ID: UC004
Actors: Member
Preconditions: 1. Member logged into the system. 2. Member filled in the membership form. 3. Member paid the registration fee through the system. 4. Member wants to join a program hold by PERWANIN.
Flow of events: 1. The system displays the commitment readiness feedback form. 2. Member fills in the commitment readiness feedback form. 3. Member submits the commitment readiness feedback form. 4. The system passes the form validation (EF1). 5. Use case ends.
Postconditions: Member's commitment readiness feedback successfully stored in the database.
Alternative flow <i>n</i>: 1. User quits to fill in commitment readiness form 1.1 The system returns to the dashboard. 1.2 The use case is aborted.
Postconditions: Member's commitment readiness feedback did not store in the database.
Exception flow (if any): 1. System failed to insert new commitment readiness feedback record into database 1.1 The system displays an error message on the screen. 1.2 Flow of events are executed again.

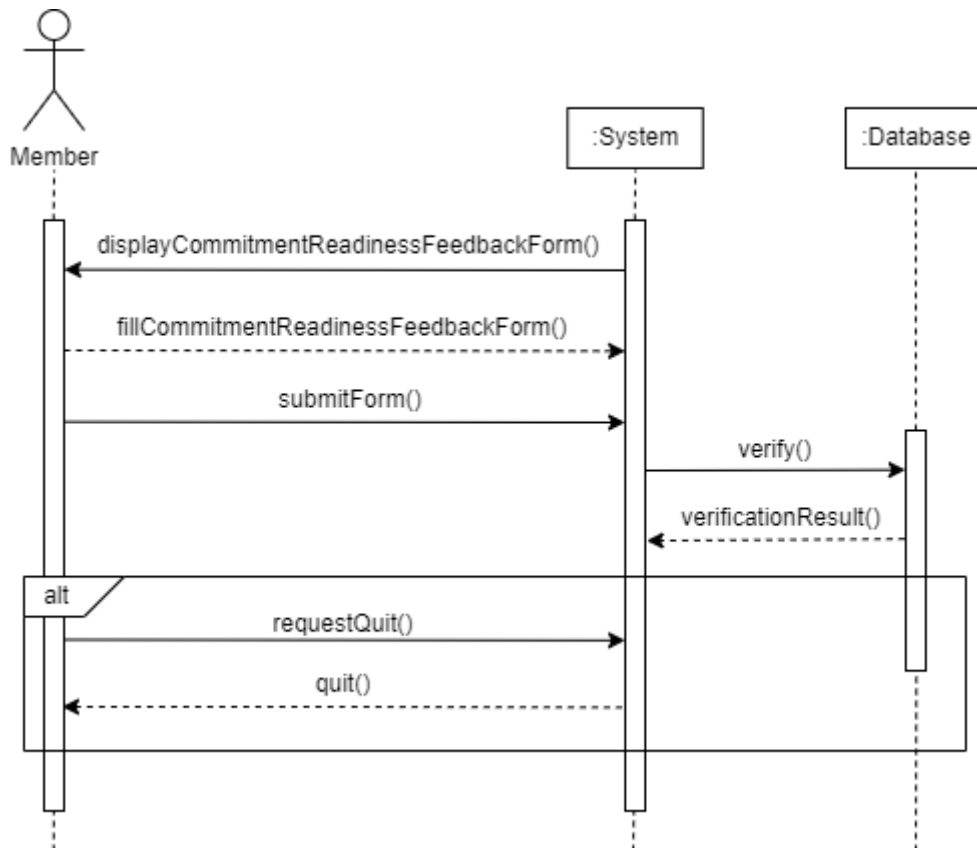


Figure 2.8: Sequence Diagram for Provide commitment readiness feedback

2.2.5 UC005: Use Case Manage member's information

Table 2.5: Use Case Description for Manage member's information

Use case: Manage member's information
ID: UC005
Actors: Secretary
Preconditions: Secretary logged into the system.
Flow of events: 1. The system displays complete list of member information. 2. Secretary chooses the action to perform. 3. Use case ends.
Postconditions: Secretary successfully updates the records of member information in the database of system.
Alternative flow <i>n</i>: 1. Edit member information

- 1.1 Secretary chooses the member information to edit.
- 1.2 The system displays editing member information form.
- 1.3 Secretary updates the member information.
- 1.4 Secretary chooses confirms and updates member information.
- 1.5 The system passes the form validation.
- 1.6 The system displays successfully updates member information.

2. Delete user

- 2.1 Secretary chooses the member information to delete.
- 2.2 Secretary chooses to delete member information.
- 2.3 The system displays successfully deletes member information.

3. Cancel operation by user

- 3.1 The system returns to the last accessed page.
- 3.2 The use case is aborted.

Postconditions:

None

Exception flow (if any):

1. Validation failed

- 1.1 The system displays an error message on the form.
- 1.2 Flow of events are executed again.

2. System failed to insert new record into database

- 2.1 The system displays an error message on the screen.
- 2.2 Flow of events are executed again.

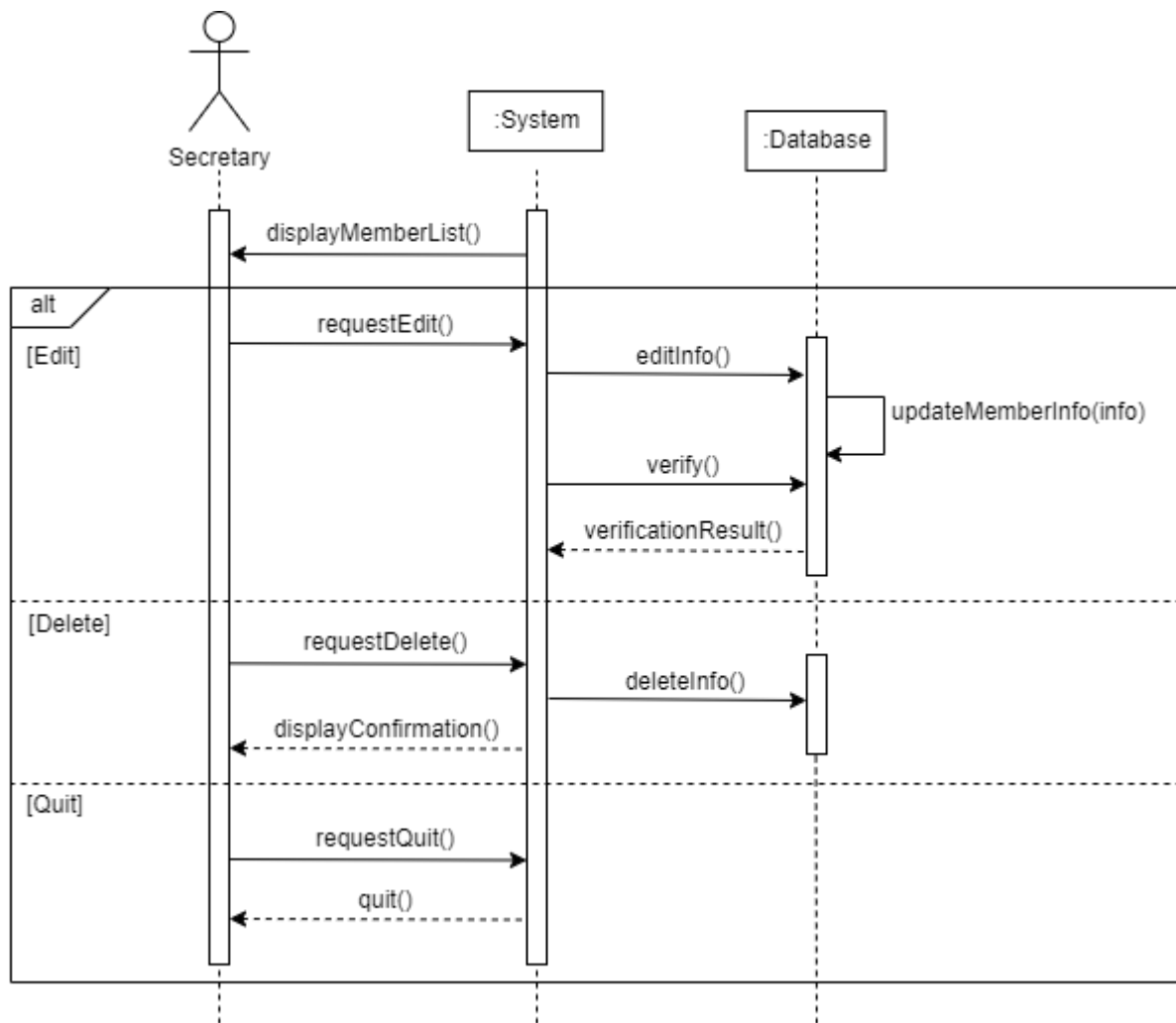


Figure 2.9: Sequence Diagram for Manage member's information

2.2.6 UC006: Use Case Search member's information

Table 2.6: Use Case Description for Search member's information

Use case: Search member's information
ID: UC006
Actors: Secretary
Preconditions: 1. Secretary logged into the system. 2. Secretary accessed into the member information database.
Flow of events: 1. The system displays a search feature. 2. Secretary enters keyword either member's ID, name, expertise or interests. 3. The system searches the member's information based on selected category.

4. The system displays the search result.
5. Use case ends.
Postconditions: Secretary successfully searches the program's information.
Alternative flow <i>n</i>: 1. Secretary quits the searching feature 1.1 The system returns to the last accessed page. 1.2 The use case is aborted. 2. Searching result failed. 2.1 The system displays no program found message on the screen. 2.2 The use case is aborted.
Postconditions: Secretary successfully searches the member's information.
Exception flow (if any): 1. System failed to search any member in the database 1.1 The system displays error messages on the screen. 1.2 Flow of events are executed again.

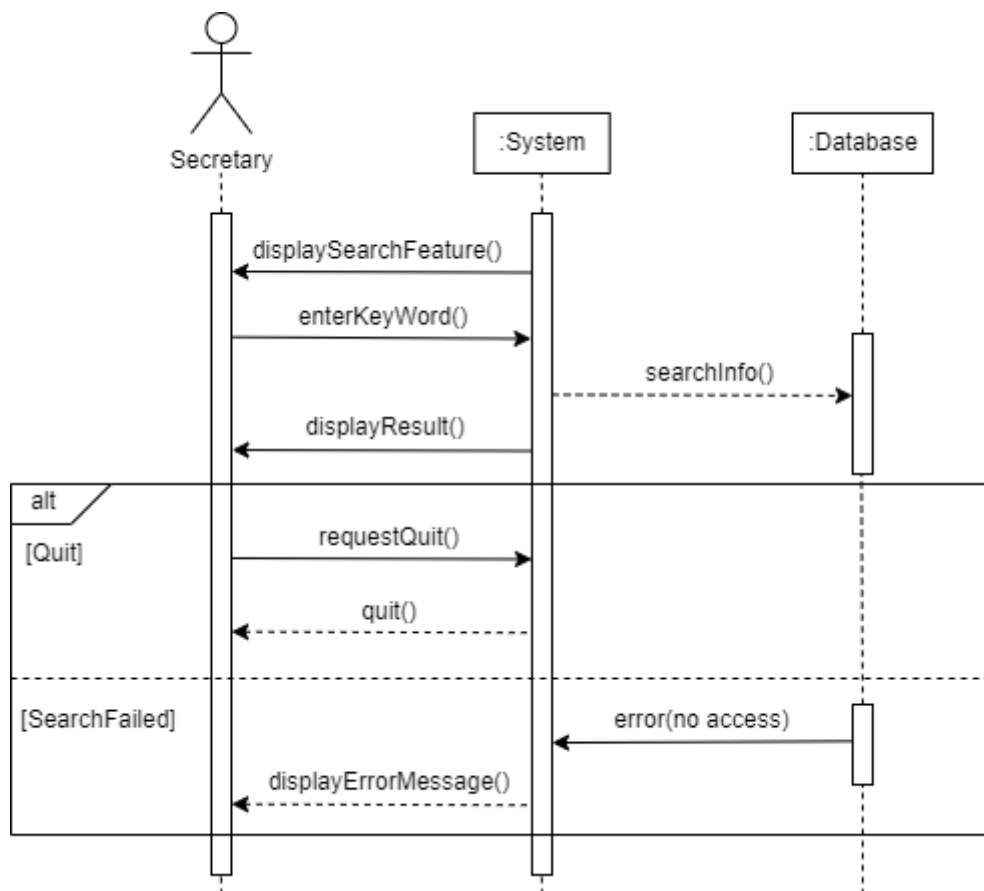


Figure 2.10: Sequence Diagram for Search member's information

2.2.7 UC007: Use Case Manage to be organized programs

Table 2.7: Use Case Description for Manage to be organized programs

Use case: Manage to be organized programs
ID: UC007
Actors: Secretary
Preconditions: 1. Secretary logged into the system. 2. Secretary accessed into the programs database.
Flow of events: 1. The system displays all the to be organized programs. 2. The secretary chooses the action to perform. 3. Use case ends.
Postconditions: The secretary successfully updates the records of to be organized programs in the system database.
Alternative flow <i>n</i>: 1. Edit to be organized programs 1.1 Secretary chooses the program to edit. 1.2 The system displays an editing program information form. 1.3 Secretary updates the program information. 1.4 Secretary chooses to confirm and update the program information. 1.5 The system passes the form validation. 1.6 The system successfully updates the program information. 2. Delete to be organized programs 2.1 Secretary chooses the program to delete. 2.2 Secretary chooses to delete the program. 2.3 The system successfully deletes the program. 3. Cancel operation by the secretary 3.1 The system returns to the last accessed page. 3.2 The use case is aborted.
Postconditions: The secretary successfully updates the records of to be organized programs in the system database.
Exception flow (if any): 1. Validation failed 1.1 The system displays an error message on the form. 1.2 Flow of events are executed again. 2. System failed to insert new records into the database 2.1 The system displays an error message on the screen. 2.2 Flow of events are executed again.

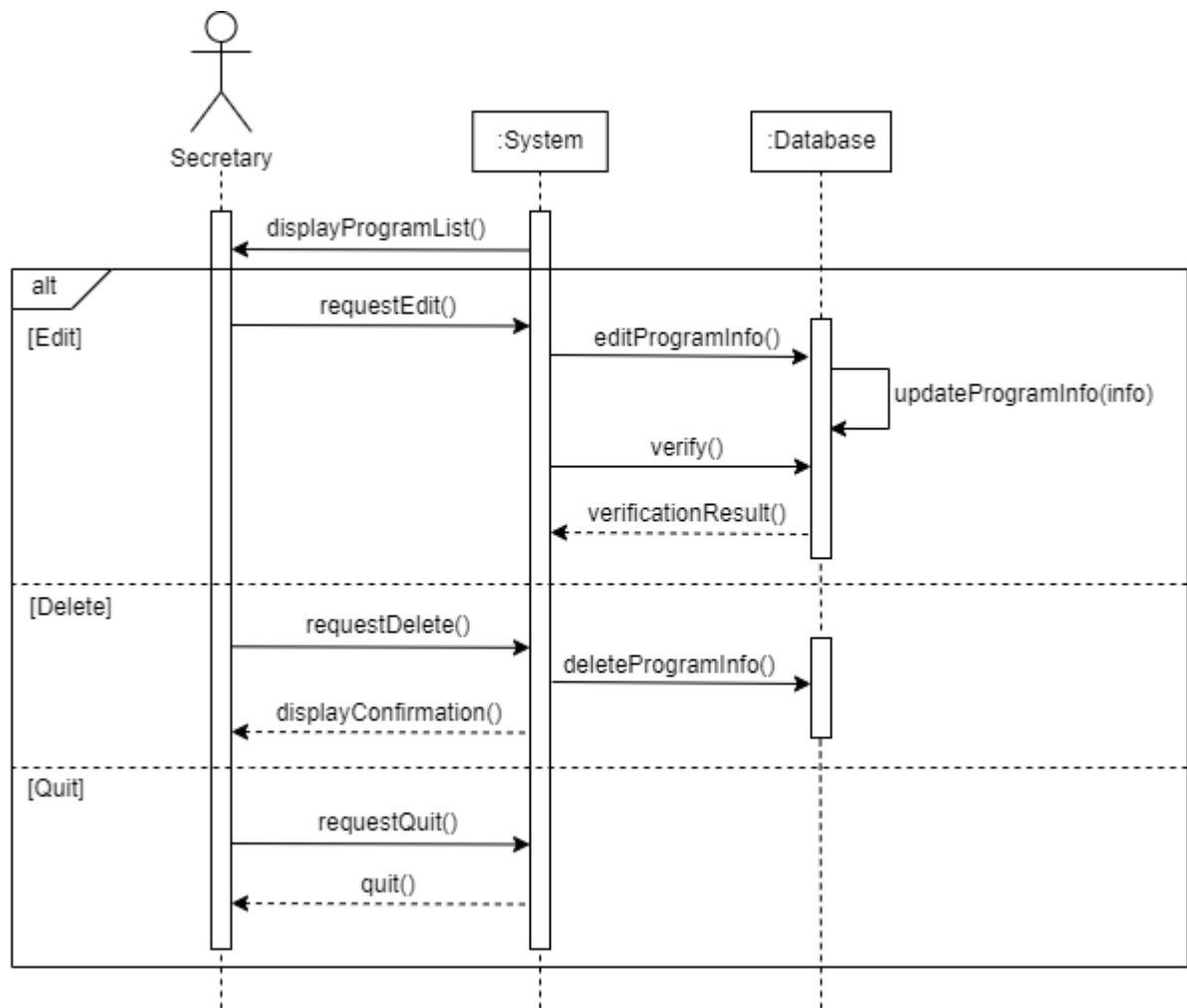


Figure 2.11: Sequence Diagram for Manage to be organized programs

2.2.8 UC008: Use Case Generate member's commitment reports

Table 2.8: Use Case Description for Generate member's commitment reports

Use case: Generate member's commitment reports
ID: UC008
Actors: Secretary
Preconditions: 1. Secretary logged into the system. 2. Secretary accessed into the member's commitment reports database.
Flow of events: 1. The system displays the list of members. 2. Secretary chooses the member in the list. 3. Secretary chooses the type of document to generate. 4. Secretary generates the member's commitment reports. 5. The system displays the member's commitment reports.

6. Use case ends.
Postconditions: The secretary successfully generates the member's commitment reports.
Alternative flow <i>n</i>: 1. Cancel operation by the secretary 1.1 The system returns to the last accessed page. 1.2 The use case is aborted.
Postconditions: The secretary successfully generates the member's commitment reports.
Exception flow (if any): 1. Generation of document failed 1.1 The system displays an error message on the document. 1.2 Flow of events are executed again.

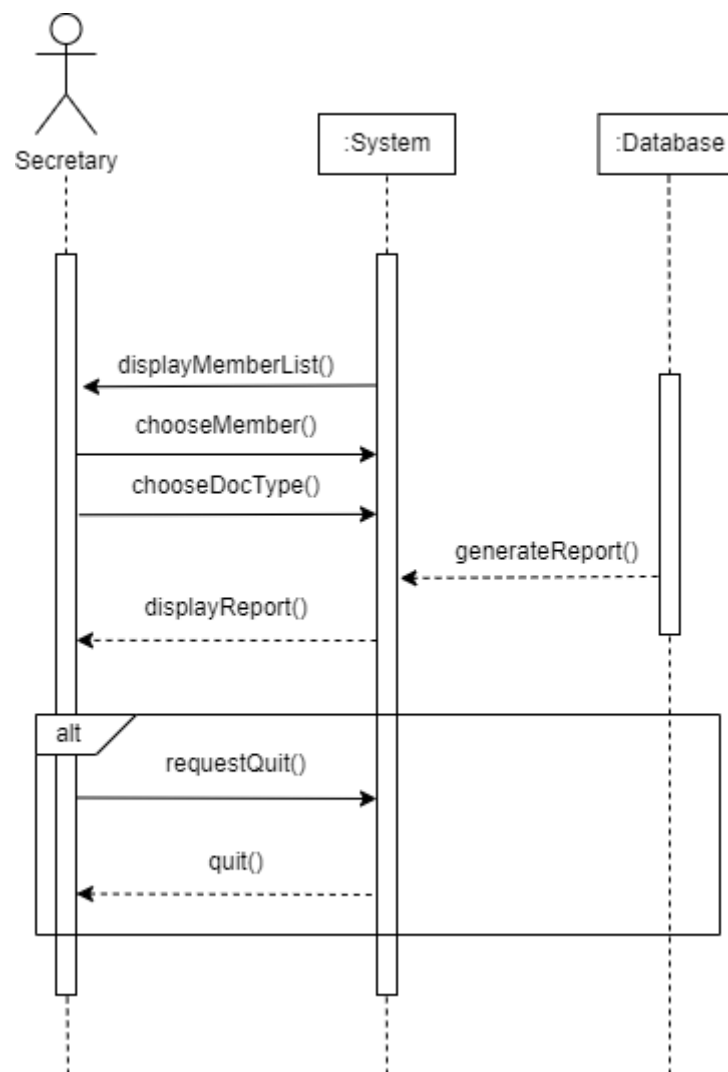


Figure 2.12: Sequence Diagram for Generate member's commitment reports

2.2.9 UC009: Use Case Manage volunteers for each program

Table 2.9: Use Case Description for Manage volunteers for each program

Use case: Manage volunteers for each program
ID: UC009
Actors: Secretary
Preconditions: 1. Secretary logged into the system. 2. Secretary accessed into the volunteer database.
Flow of events: 1. The system displays all the volunteers. 2. The secretary chooses the action to perform. 3. Use case ends.
Postconditions: The secretary successfully updates the records of volunteers for each program in the volunteer database.
Alternative flow <i>n</i>: 1. Edit volunteer's information 1.1 Secretary chooses the volunteer to edit. 1.2 The system displays an editing volunteer information form. 1.3 Secretary updates the volunteer information. 1.4 Secretary chooses to confirm and update the volunteer information. 1.5 The system passes the form validation. 1.6 The system successfully updates the volunteer information. 2. Delete volunteer's information 2.1 Secretary chooses the volunteer to delete. 2.2 Secretary chooses to delete the volunteer's information. 2.3 The system successfully deletes the volunteer's information. 3. Cancel operation by the secretary 3.1 The system returns to the last accessed page. 3.2 The use case is aborted.
Postconditions: The secretary successfully updates the records of volunteers for each program in the volunteer database.
Exception flow (if any): 1. Validation failed 1.1 The system displays an error message on the form. 1.2 Flow of events are executed again. 2. System failed to insert new records into the database 2.1 The system displays an error message on the screen. 2.2 Flow of events are executed again.

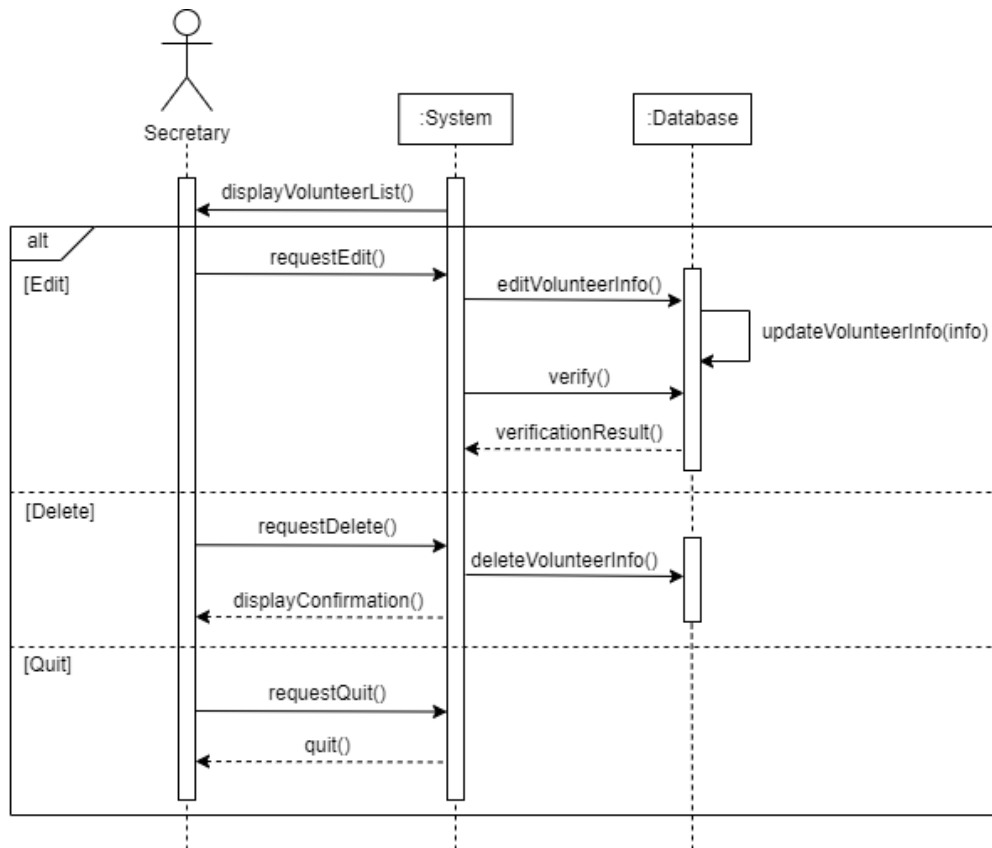


Figure 2.13: Sequence Diagram for Manage volunteers for each program

2.2.10 UC010: Use Case Search for volunteers

Table 2.10: Use Case Description for Search for volunteers

Use case: Search for volunteers
ID: UC010
Actors: Secretary
Preconditions: 1. Secretary logged into the system. 2. Secretary accessed into the volunteer database.
Flow of events: 1. The system displays a search feature. 2. Secretary enters either volunteer's expertise, interests or location of volunteers. 3. The system searches the volunteer's information based on the selected category. 4. The system displays the search result. 5. Use case ends.
Postconditions: The secretary successfully searches the program's information.
Alternative flow <i>n</i>:

1. Cancel the operation

- 1.1 The system returns to the last accessed page.
- 1.2 The use case is aborted.

2. Search result is not displayed

- 2.1 The system displays error messages on the screen.
- 2.2 The use case is aborted.

Postconditions:

The secretary successfully searches the program's information.

Exception flow (if any):

1. System failed to search the result

- 1.1 The system displays an error message on the screen.
- 1.2 Flow of events are executed again.

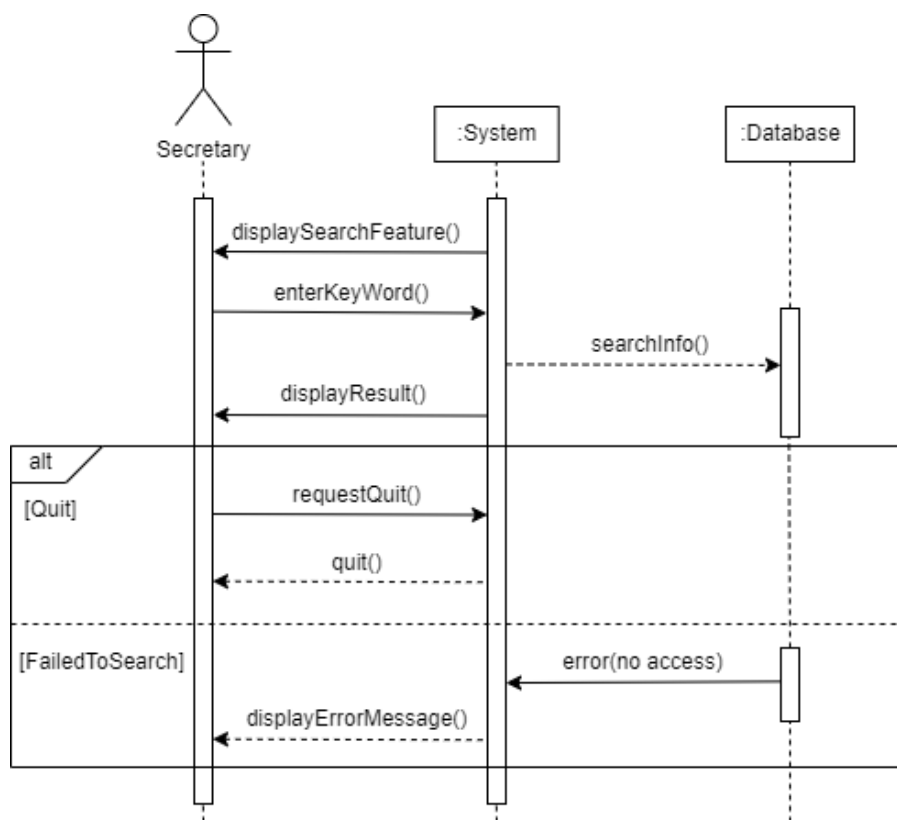


Figure 2.14: Sequence Diagram for Search for volunteers

2.2.11 UC011: Use Case Send invitation notice

Table 2.11: Use Case Description for Send invitation notice

Use case: Send invitation notice
ID: UC011
Actors: Secretary
Preconditions: 1. Secretary logged into the system.
Flow of events: 1. The system displays the list of the to be organized programs. 2. Secretary chooses the program to send the notice. 3. The system displays the list of the volunteers. 4. Secretary chooses the volunteers to send the notice. 5. Secretary sends the invitation notice to the suitable volunteers. 6. Use case ends.
Postconditions: The invitation notice is successfully sent out by secretary.
Alternative flow <i>n</i>: 1. Cancel the operation 1.1 The system returns to the last accessed page. 1.2 The use case is aborted.
Postconditions: The invitation notice is successfully sent out by secretary.
Exception flow (if any): 1. System failed to send out the notice 1.1 The system displays an error message on the screen. 1.2 Flow of events are executed again.

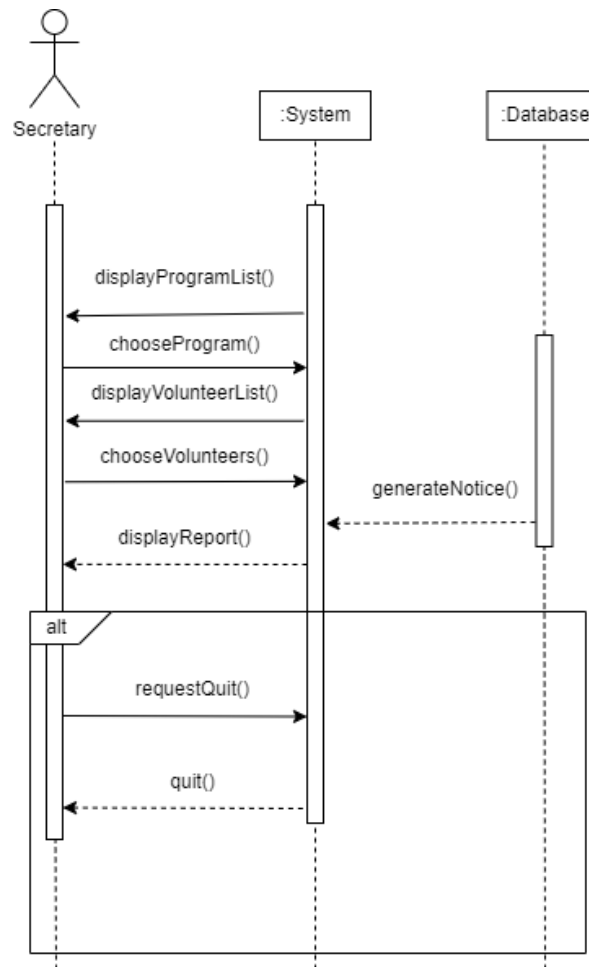


Figure 2.15: Sequence Diagram for Send invitation notice

2.2.12 UC012: Use Case Search program's information

Table 2.12: Use Case Description for Search program's information

Use case: Search program's information
ID: UC012
Actors: Volunteer
Preconditions: Volunteers logged into the system.
Flow of events: 1. Volunteer enters the related program name or keyword. 2. The system searches in database. 3. The system displays the search result on the screen. 4. Use case ends.
Postconditions: Volunteer successfully search the program's information.
Alternative flow <i>n</i>:

1. Volunteer quits the searching feature

1.1 The system returns to the last accessed page.

1.2 The use case is aborted.

2. Searching result failed.

2.1 The system displays no program found message on the screen.

2.2 The use case is aborted.

Postconditions:

Volunteer successfully searches the program's information.

Exception flow (if any):

1. System failed to search any program in the database

1.1 The system displays error messages on the screen.

1.2 Flow of events are executed again.

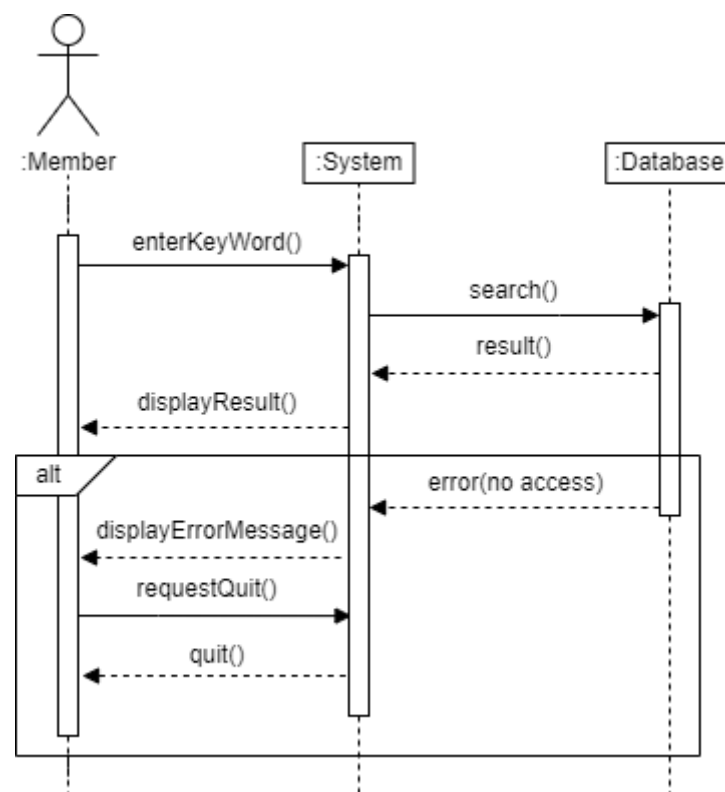


Figure 2.16: Sequence Diagram for Search program's information

2.2.13 UC013: Use Case Receive invitation notice

Table 2.13: Use Case Description for Receive invitation notice

Use case: Receive invitation notice
ID: UC013
Actors: Volunteer
Preconditions: Volunteers logged in into the system.
Flow of events: 1. Volunteers receive the invitation notice. 2. The system displays invitation notices on the screen. 3. Use case ends.
Postconditions: Volunteer successfully views the invitation notice.
Alternative flow <i>n</i>: 1. Volunteer quits the invitation notice 1.1 The system returns to the last accessed page. 1.2 The use case is aborted. 2. Invitation notice failed 2.1 The system displays no invitation notice found message on the screen. 2.2 The use case is aborted.
Postconditions: None
Exception flow (if any): None

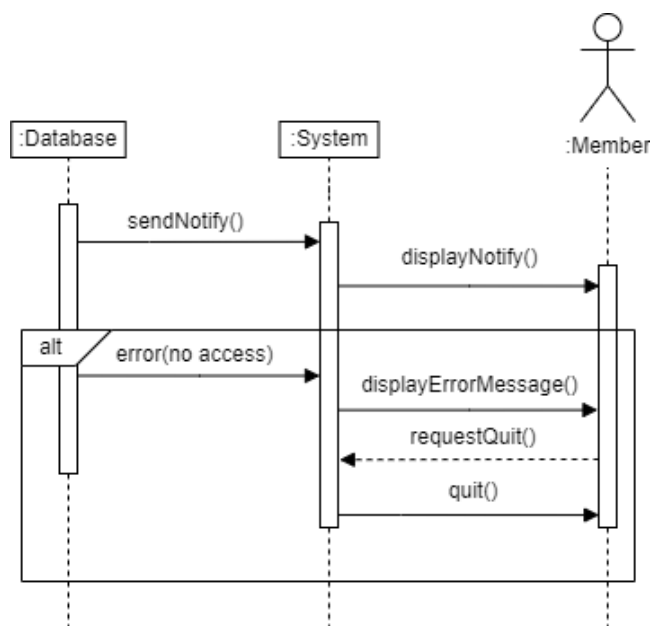


Figure 2.17: Sequence Diagram for Receive invitation notice

2.2.14 UC014: Use Case Access program information

Table 2.14: Use Case Description for Access program information

Use case: Access program information
ID: UC014
Actors: Volunteer and donors
Preconditions: Donors logged in into the system.
Flow of events: 1. User chooses the program information. 2. The system displays the program information. 3. Use case ends.
Postconditions: User successfully accesses the program information.
Alternative flow <i>n</i>: 1. User quits to access the program information. 1.1 The system returns to the last accessed page. 1.2 The use case is aborted. 2. Access program information failed. 2.1 The system displays no program information found message on the screen. 2.2 The use case is aborted.
Postconditions: None
Exception flow (if any): None

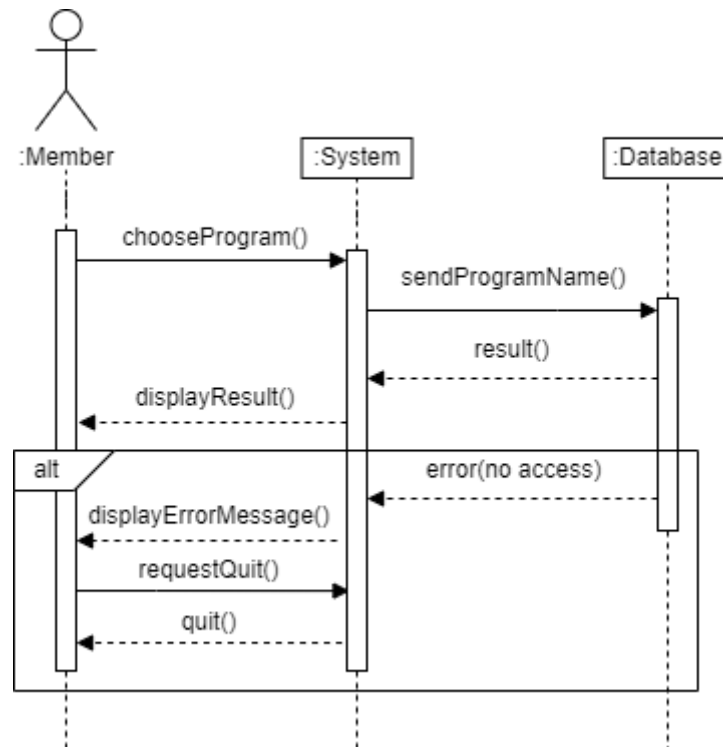


Figure 2.18: Sequence Diagram for Access program information

2.2.15 UC015: Use Case Make donations

Table 2.15: Use Case Description for Make donations

Use case: Make donations
ID: UC015
Actors: Donors
Preconditions: Donors logged in into the system.
Flow of events: 1. The system displays the donation form. 2. Donors enters the donation information. 3. Donors chooses confirm the donation information. 4. The system passes form validation. 5. The system performs database insert and update. 6. System successfully inserted a new donation into the database. 7. Use case ends.
Postconditions: Donors successfully make donations into the system.
Alternative flow <i>n</i>: 1. Cancel donations by donors 1.1 The system returns to the last accessed page.

1.2 The use case is aborted.

Postconditions:

None

Exception flow (if any):

1.Validation failed

1.1 The system displays an error message on the form.

1.2 Flow of events are executed again.

2. System failed to insert new donation record into database

2.1 The system displays error messages on the screen.

2,2 Flow of events are executed again.

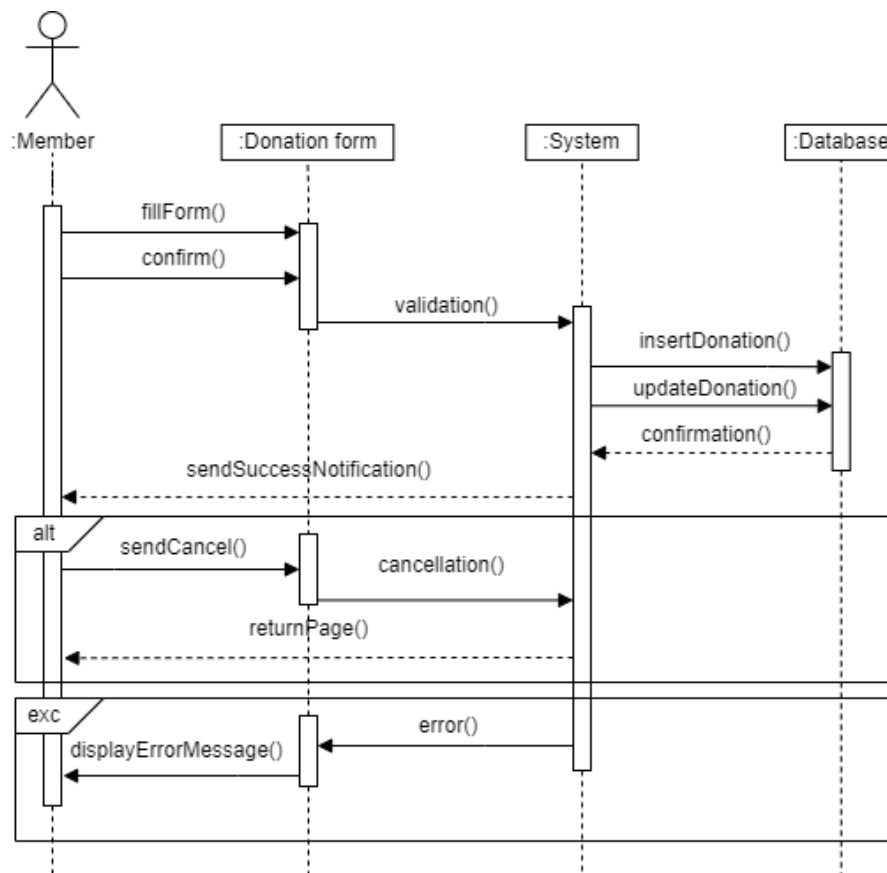


Figure 2.19: Sequence Diagram for Make donations

2.2.16 UC016: Use Case Manage financial information

Table 2.16: Use Case Description for Manage financial information

Use case: Manage financial information
ID: UC016
Actors: Treasurer
Preconditions: Treasurer logged into the system.
Flow of events: 1. The system displays complete list of financial information. 2. Treasurer chooses the action to perform. 3. Use case ends.
Postconditions: Treasurer successfully updates the records of financial information in the database of system.
Alternative flow <i>n</i>: 1. Edit financial information 1.1 Treasurer chooses the financial information to edit. 1.2 The system displays editing financial information form. 1.3 Treasurer updates the financial information. 1.4 Treasurer chooses confirm and updates financial information. 1.5 The system passes the form validation. 1.6 The system successfully updates financial information. 2. Delete user 2.1 Treasurer chooses the financial information to delete. 2.2 Treasurer chooses to delete financial information. 2.3 The system successfully deletes financial information. 3. Cancel operation by user 3.1 The system returns to the last accessed page. 3.2 The use case is aborted.
Postconditions: Treasurer successfully updates the records of financial information in the database of system.
Exception flow (if any): 1. Validation failed 1.1 The system displays an error message on the form. 1.2 Flow of events are executed again. 2. System failed to insert new record into database 2.1 The system displays an error message on the screen. 2.2 Flow of events are executed again.

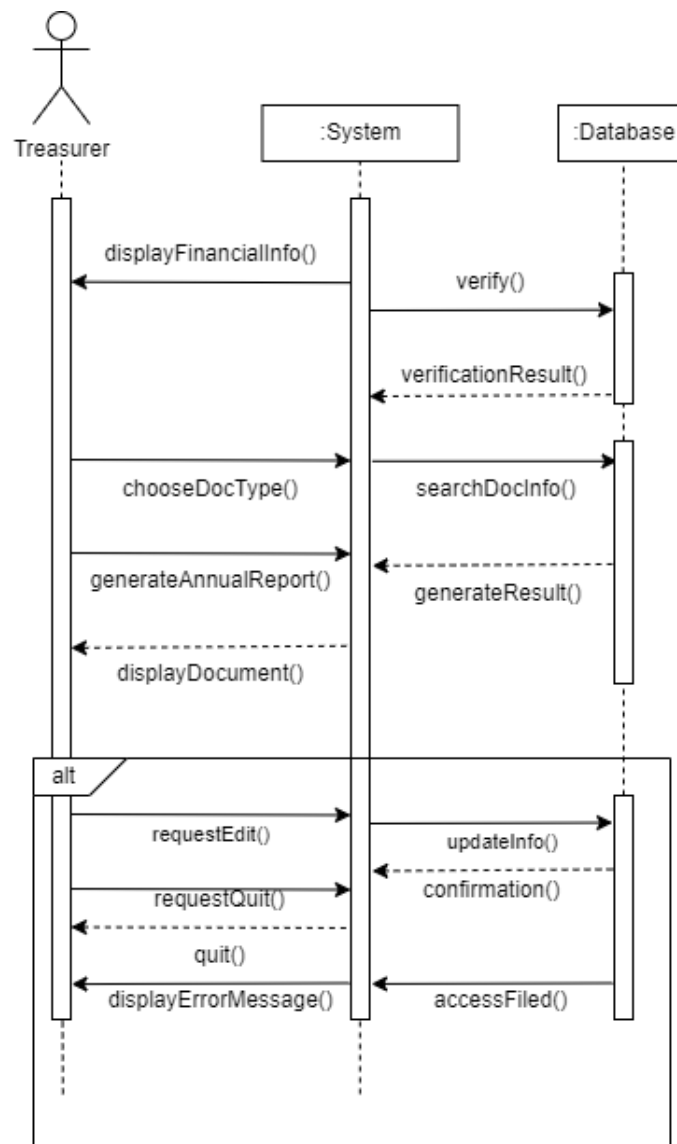


Figure 2.20: Sequence Diagram for Manage financial information

2.2.17 UC017: Use Case Generate financial documents

Table 2.17: Use Case Description for Generate financial documents

Use case: Generate financial documents
ID: UC017
Actors: Treasurer
Preconditions: 1. Treasurer logged into the system. 2. Treasurer accessed into the financial information.
Flow of events: 1. System displays the financial information. 2. Treasurer chooses the type of document be generated.

3. Treasurer requests filtering data in system.
4. System displays data after filtering.
5. Treasurer uses filtering data to generate desired document.
6. Use case ends.

Postconditions:

Treasurer successfully generates financial document.

Alternative flow *n*:

1. Cancel generate operation by Treasurer

- 1.1 The system returns back to the last accessed page.
- 1.2 The use case is aborted.

Postconditions:

Treasurer successfully generates financial document.

Exception flow (if any):

1. Document generation failed

- 1.1 The system displays error message on the document.
- 1.2 NF2 is executed again.

2. System failed to filter data in database

- 2.1 The system displays error message on the screen.
- 2.2 NF3 is executed again.

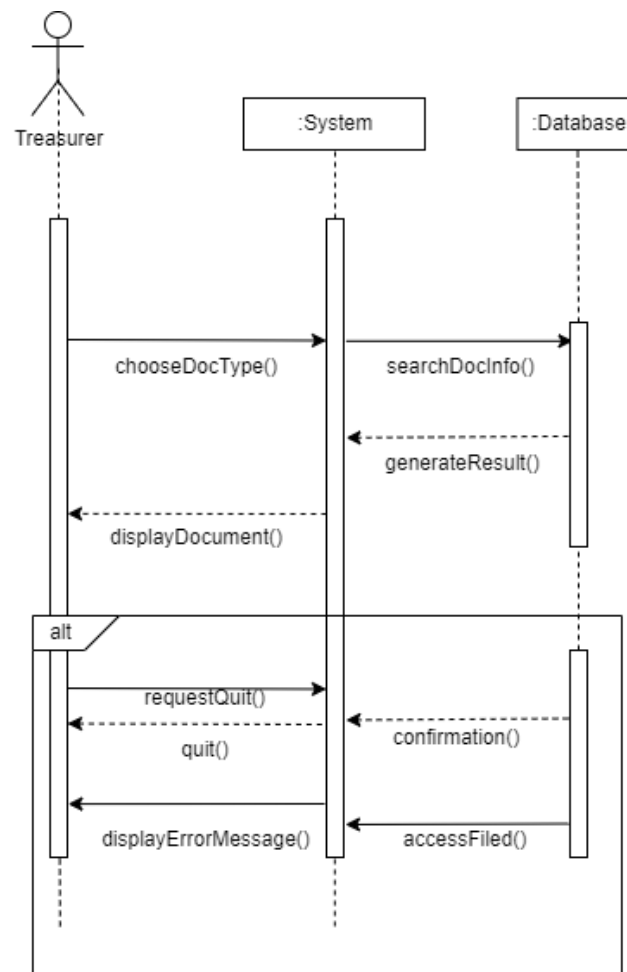


Figure 2.21: Sequence Diagram for Generate financial documents

2.2.18 UC018: Use Case Generate vouchers

Table 2.18: Use Case Description for Generate vouchers

Use case: Generate vouchers
ID: UC018
Actors: Treasurer
Preconditions: 1. Treasurer logged in into the system. 2. Treasurer accessed into the information data.
Flow of events: 1. System displays the list of voucher information. 2. Treasurer chooses to generate vouchers. 3. System filters the information related to vouchers. 4. System displays data after filtering. 5. Treasurer uses filtering data to generate vouchers. 6. Use case ends.
Postconditions: Treasurer successfully generates vouchers.
Alternative flow <i>n</i>: 1. Change type of document be generated 1.1 The system return back to the page list all categories (NF1). 1.2 Treasurer chooses the type of document want to generate. 1.3 System navigates to correct page. 2. Cancel operation by treasture 2.1 The system returns back to the last accessed page. 2.2 The use case is aborted.
Postconditions: Treasurer successfully generates vouchers.
Exception flow (if any): 1. Document generation failed 1.1 The system displays error message on the document. 1.2 NF2 is executed again. 2. System failed to Filtering data in database 2.1 The system displays error message on the screen. 2.2 NF3 is executed again.

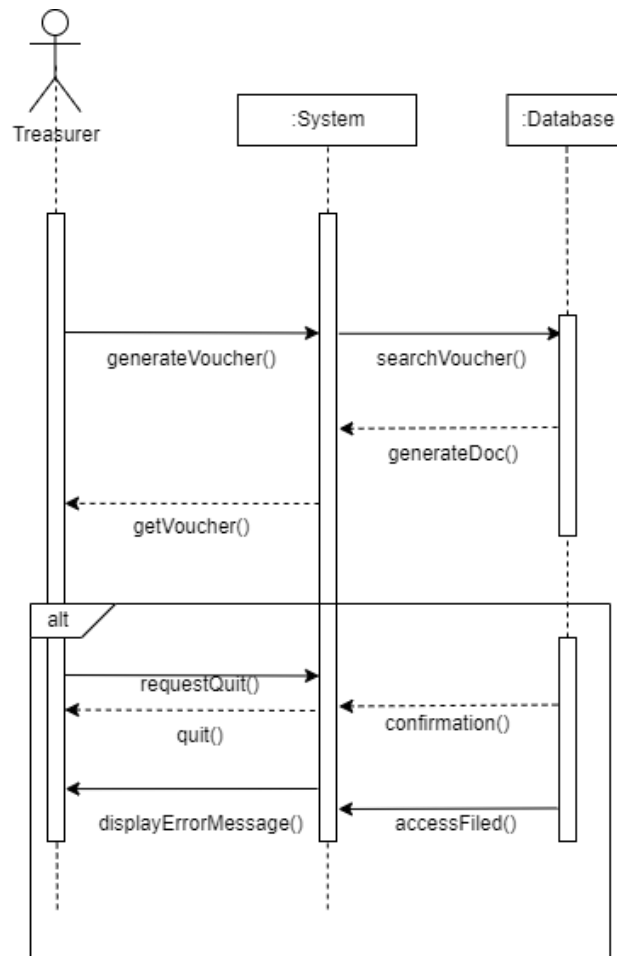


Figure 2.22: Sequence Diagram for Generate vouchers

2.2.19 UC019: Use Case Generate financial report

Table 2.19: Use Case Description for Generate financial report

Use case: Generate financial report
ID: UC019
Actors: Treasurer
Preconditions: 1. Treasurer logged in into the system. 2. Treasurer accessed into the information data.
Flow of events: 1. System lists the categories of document can be generated. 2. Treasurer chooses to generate financial report. 3. System filters the financial information. 4. System displays data after analyzing. 5. Treasurer uses the analyzed data to generate financial report. 6. Use case ends.

Postconditions:

Treasurer successful generates financial report.

Alternative flow *n*:**1. Change type of document be generated**

- 1.1 The system returns back to the page list all categories (NF1).
- 1.2 Treasurer chooses the type of document want to generate.
- 1.3 System navigates to correct page.

2. Cancel operation by treasurer

- 2.1 The system returns back to the last accessed page.
- 2.2 The use case is aborted.

Postconditions:

Treasurer successful generate financial report.

Exception flow (if any):**1. Document generation failed**

- 1.1 The system displays error message on the document.
- 1.2 NF2 is executed again.

2. System failed to Filtering data in database

- 2.1 The system displays error message on the screen.
- 2.2 NF3 is executed again.

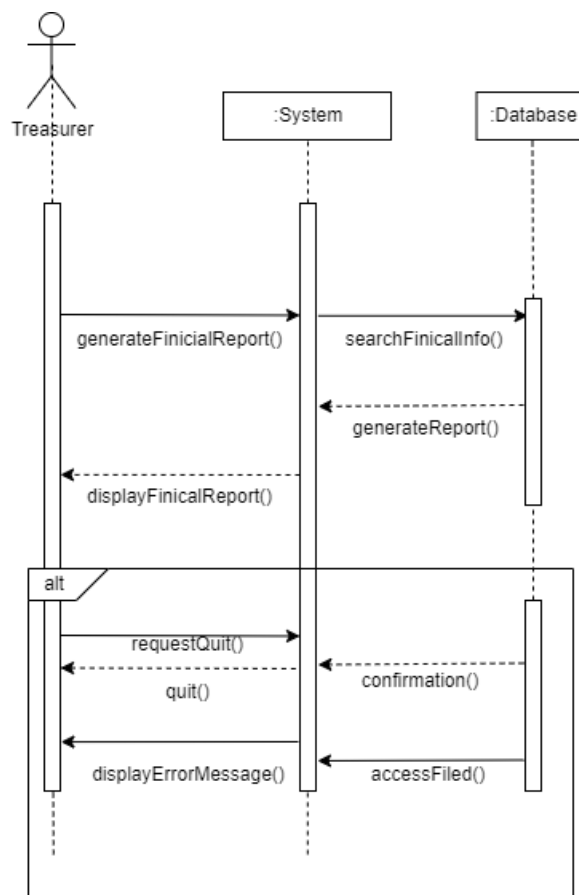


Figure 2.23: Sequence Diagram for Generate financial report

2.2.20 UC020: Use Case Generate receipt

Table 2.20: Use Case Description for Generate receipt

Use case: Generate receipt
ID: UC020
Actors: Treasurer
Preconditions: 1. Treasurer logged into the system. 2. Treasurer accessed into the information data.
Flow of events: 1. System lists down the payment information. 2. Treasurer chooses to generate receipt. 3. System filters and correct the payment information. 4. System displays data after analyzing. 5. Treasurer uses analyzed data to generate receipt. 6. Use case ends.
Postconditions: Treasurer successful generates the receipt.
Alternative flow <i>n</i>: 1. Change type of document be generated 1.1 The system return back to the page list all categories (NF1). 1.2 Treasurer choose the type of document want to generate. 1.3 System navigates to correct page. 2. Cancel operation by treasurer 2.1 The system returns back to the last accessed page. 2.2 The use case is aborted.
Postconditions: Treasurer successful generates the receipt.
Exception flow (if any): 1. Document generation failed 1.1 The system displays error message on the document. 1.2 NF2 is executed again. 2. System failed to Filtering data in database 2.1 The system displays error message on the screen. 2.2 NF3 is executed again.

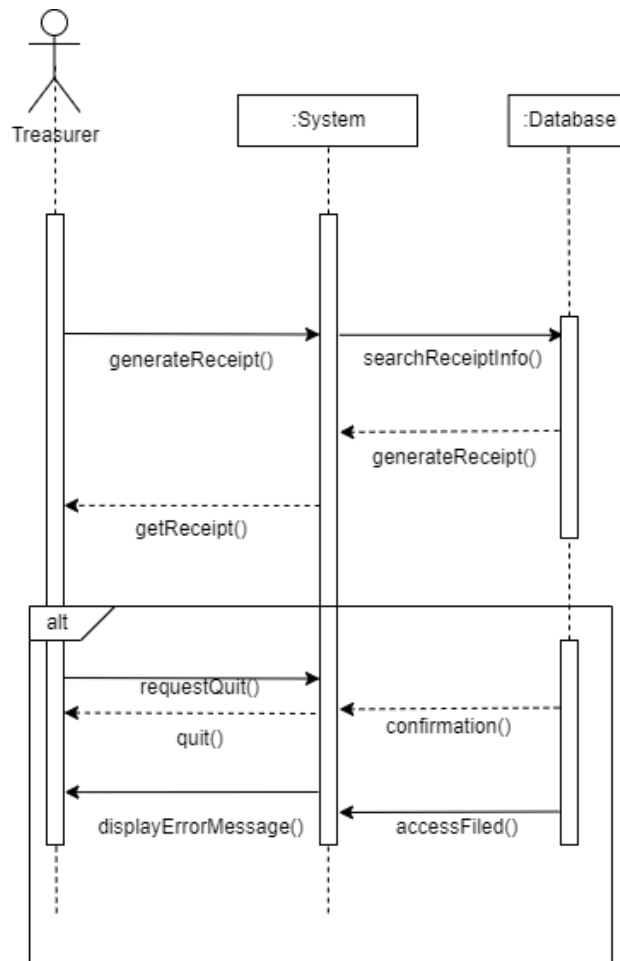


Figure 2.24: Sequence Diagram for Generate receipt

2.2.21 UC021: Use Case Manage donations

Table 2.21: Use Case Description for Manage donations

Use case: Manage donations
ID: UC021
Actors: Treasurer
Preconditions: 1. Treasurer logged into the system. 2. Treasurer accessed into donation system. 3. Donors accessed donation program information through Perwanin website.
Flow of events: 1. System provides payment mechanisms to donors. 2. System receives donation info of donors. 3. System displays manage option to Treasurer. 4. Treasurer generates receipt for Donors. 5. System monitoring the progress.

6. Use case ends. 7. System displays collection progress for programs by dashboard.
Postconditions: Treasurer successfully views collection progress for program.
Alternative flow <i>n</i>: 1. Delete donation record 1.1 Treasurer chooses the donation record to delete. 1.2 Treasurer chooses to delete the donation. 1.3 The system successfully deletes the record. 2. Cancel operation by user 2.1 The system returns back to the last accessed page. 2.2 The use case is aborted.
Postconditions: Treasurer successfully views collection progress for program.
Exception flow (if any): 1. Document generation failed 1.1 The system displays error message on the document. 1.2 NF4 is executed again. 2. Monitor progress failed 2.1 The system display error message on the screen. 2.2 NF5 is executed again.

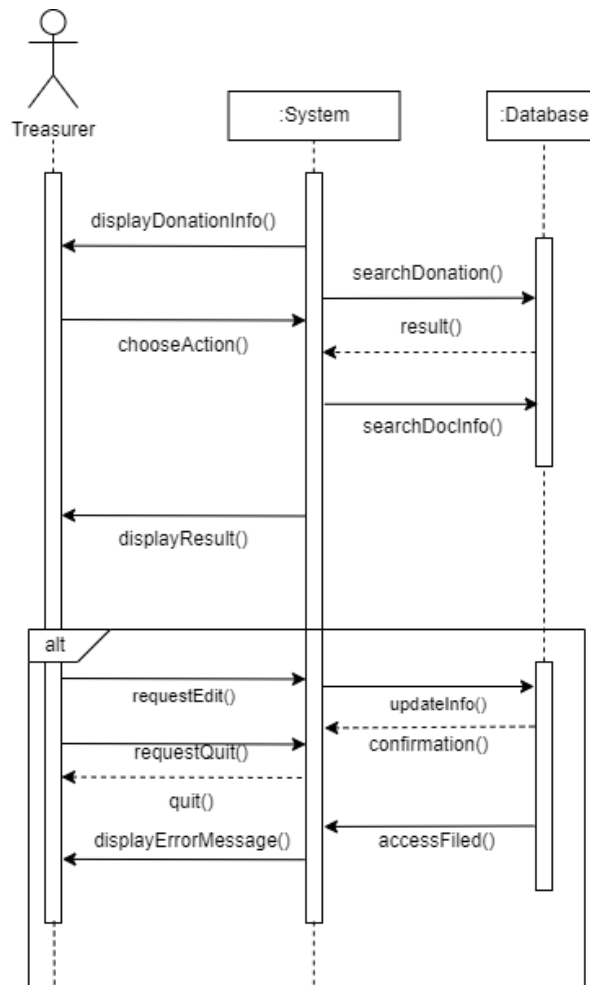


Figure 2.25: Sequence Diagram for Manage donations

2.2.22 UC022: Use Case Show collection progress

Table 2.22: Use Case Description for Show collection program

Use case: Show collection program
ID: UC022
Actors: Treasurer
Preconditions: 1. Treasurer logged into Dashboard. 2. System accessed the program data.
Flow of events: 1. The system displays Program progress information in dashboard. 2. Treasurer chooses the program to view. 3. The system displays corresponding program progress. 4. Use case ends.

Postconditions:

The system successful displays the correct program progress.

Alternative flow *n*:**1. Change another program to view**

1.1 The system returns back to the dashboard.

1.2 NF2 is executed again.

2. Cancel operation by user

2.1 The system returns back to the last accessed page.

2.2 The use case is aborted.

Postconditions:

The system successful displays the correct program progress.

Exception flow (if any):**1. Dashboard display failed**

1.1 The system displays error message on the screen.

1.2 NF1 is executed again.

2. Program progress displayed incorrect

2.1 The system displays error message on the dashboard.

2.2 NF3 is executed again.

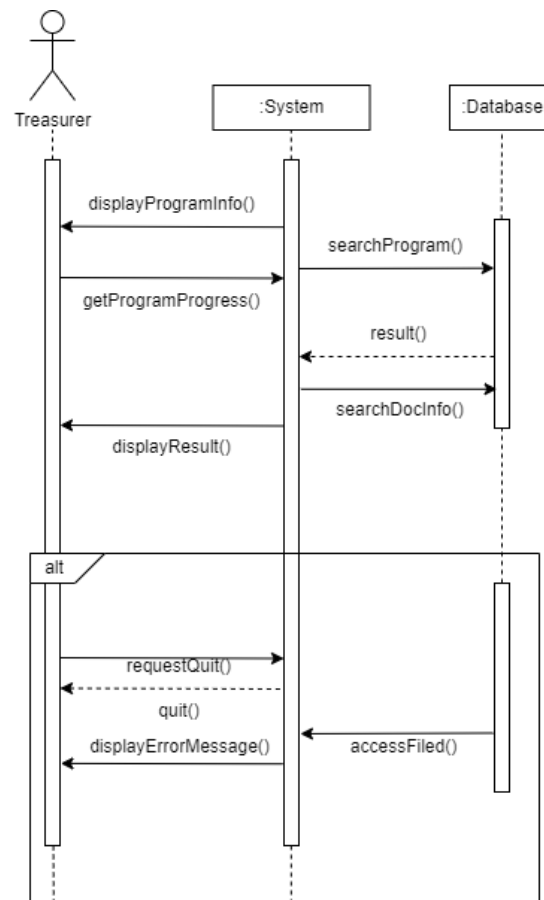


Figure 2.26: Sequence Diagram for Show collection program

2.3 Performance and Other Requirements

- The system shall be able to operate 24/7.
- The system is connected to the internet.
- The notification of system will last 5 seconds.
- The load time and verification shall take no longer than 3 seconds.
- Changes made in the database should be updated instantly.
- Average response time per interface must be less than 3 seconds.

2.4 Design Constraints

- a) All user information will be protected under the Personal Data Protection Act 2010 (PDPA).
- b) All data must be securely stored in the database.
- c) Certain documents can only be accessed by certain user roles only.
- d) The entire system must comply with the laws of Malaysia.
- e) The database must have its own security level for all data.

2.5 Software System Attributes

- The system should use a universal language to allow a better understanding by the user.
- The user only needs less than 5 hours of training and surfing through the system to have a full understanding of the system.
- The system interfaces must be easy to use by the user in any age range especially the disabled, the poor, single mothers, orphans, disaster victims, and other groups in need.
- The system should be enclosed to keep the privacy of the users such as members, volunteers and donors.
- A valid admin has to log in to the system to manage the data in the system.